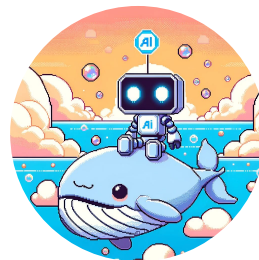




【Harness Engineering入門】AIエージェントを制御するアプローチ

ハーネスエンジニアリングにどう向き合うか ～ルールファイルを超えて開発プロセスを設計する～

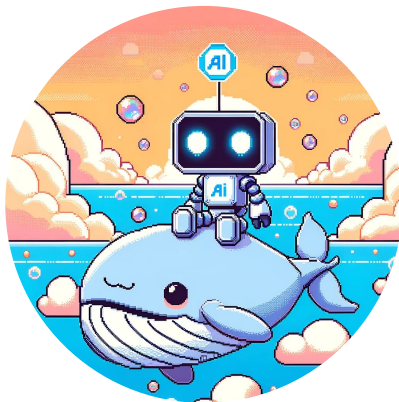
2026年4月22日
Asterminds株式会社 r.kagaya



自己紹介

r.kagaya(@ry0_kaga)

Asterminds(アスターマインズ)株式会社 共同創業者・CTO



2022年に株式会社ログラスに入社
経営管理SaaSの開発、開発生産性向上に取り組んだのち、
生成AI/LLMチームを立ち上げ、新規AIプロダクトの立ち
上げに従事、その後、25年8月に独立・現職

翻訳を担当したAIエンジニアリングが
オライリージャパンより出版



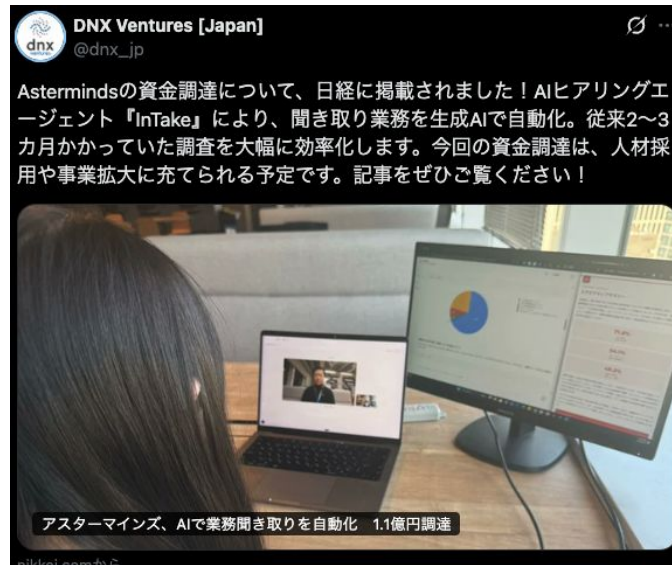
先月3月にシードラウンド調達を公開(創業から約半年)

* Asterminds

シードラウンドで
累計資金調達 **1.1億円**



delight ventures



創業半年のシードスタートアップで
ハーネスエンジニアリング(的なもの)に
どう向き合ったか？

前提

創業からシード調達までの半年弱は、1人でプロダクト開発を担っていました
その中でAIネイティブな開発プロセスを探求して、試行錯誤してきた内容です

「今の時代にゼロから立ち上げる開発組織/プロセスはどうあるべきか？」の考え
で試行錯誤した結果のため、世間一般的なハーネスエンジニアリングに該当する
ものもあれば、ハーネスより上位レイヤーに分類すべき取り組みも含まれます

組織やプロセス全体で、どう「エージェントによる開発」を行うかが中心です

ハーネスエンジニアリングの難しさ

ハーネスエンジニアリングの難しさ

昨今特有の曖昧さによる捉え所のなさ、寿命・有効範囲の見極めの難しさ

何をすべきかわからない

LangChainは「モデルでなければハーネス」と呼び、Anthropicは「ツールコールをルーティングする制御ループ」と言う

ベンダーと利用者で定義やスコープも違う(エージェントハーネス/ユーザーハーネス)

効いてるかわからない

「このハーネスは効いている」と言い切るための物差しまでではないことが多い

結果的にルールファイルでも起きた定量的・実験的・継続的な改善活動を回すのが手探りな状態な印象

いつ消えるかわからない

モデル / モデルプロバイダーによる進化に影響を受ける

モデル進化で不要な制御や処理になる可能性に加えて、Claude Managed Agentsの登場は今後も起きうる

ハーネスは何で構成されるのか？

利用者側であるなら、記憶と文脈、品質と安全がメインのスコープ？
system prompts、[CLAUDE.md](#)、skills、tools、orchestration、
hooks、observabilityなどが中心

カテゴリ	何を（コンポーネント）	どう（パターン）
記憶と文脈	メモリ、コンテキスト管理、プロンプト構築	永続ルールファイル、スコープ別読み込み、3層メモリ、段階的圧縮
制御と実行	制御ループ、ツール、状態管理、エラー処理	探索→計画→実行ループ、並列分岐、目的特化ツール
品質と安全	ガードレール、検証ループ	リスク事前分類、ルール/目視/LLM判定、3層ガード
拡張と自動化	サブエージェント、ライフサイクル管理	段階的ツール拡張、決定論的フック、コンテキスト分離

ハーネスエンジニアリングで何を達成したいか？

「エージェントが動く環境・システムの設計」

事前の予防(フィードフォワード)と事後の検証(フィードバック)を設計する

行動を制約する

- ・何をどう作るかを定める
- ・解空間を絞る

CLAUDE.md、スキル定義、
ワークフロー、spec.md

正しいか確かめる

- ・検証し、問題があれば差し戻す

静的チェック、ブラウザテスト、探索的テスト、AIレビュー

次をもっと良くする

学びを蓄積する仕組み

Self-Refinement、メモリ、実行ログ分析、ハーネスの自動修正、パターンDB

テストは事後の検出、ハーネスで事前の予防と、失敗からの学習まで含む

ハーネスエンジニアリングの難しさ

ハーネスエンジニアリングにおいても、ルール/スキル整備と個人の感性による「俺の考える最強のX」に留まっていないか？

ルールファイルやスキルの整備は一部

ルールやスキルは強力。一方でルールファイルやスキルを整備するだけがハーネスエンジニアリングだとは思わない
ソフトウェアエンジニアリングとして向き合える領域

評価・Evalこそが長期的な改善を導く

ルールファイル/スキルの時代から、「その修正は何にどう/どの程度Hitするのか？」をわかることの重要性
ハーネスを育てるにも実行/思考履歴と評価基準

ターミナルの外へ、個人の活用を超えて

Claude Code/Codexを配り、ルールファイルを整備すればAIネイティブな開発か？
「個人がどれだけうまく使えるか」に閉じていないか？

一番好きな捉え方は、「モデルでなければハーネス」

ワークフロー、パイプライン、評価、組織、顧客接点まで、
モデルの外側は全部ハーネスの範囲として、発想が広がる
(これはこれで広すぎるので全てがハーネスになるが)

Claude Code/Codexを配り、ルールファイルを整備すればAIネイティブな開発か？

少なくともプロダクト組織・プロセス全体はNO

「個人がどれだけうまく使えるか」に閉じてしまいがち(印象)

ハーネスの定義論はさておくと、ソフトウェア開発に従事する身で考えることは、
コーディングエージェントで開発するだけでなく、コーディングエージェントが機能
する環境や開発プロセスを設計すること

ハーネスエンジニアリングもあるし、上位に位置しそうなものもある
環境や開発プロセスの捉え方を広げたら、いわゆるユーザーハーネスの範疇を超
えて取り組めることはたくさんある

实践

ハーネスエンジニアリング(関連)の取り組み例

ハーネスエンジニアリング(に近い)取り組みの中で、比較的被らなさそうな下記3点を主に取り上げます

決定論と確率論の
開発パイプライン

Claude Agent SDK でパイプラインのコード・スクリプト化
決定論的ステップ(ex. lint、ブランチpush)とエージェント的自由(実装、CI修正)を交互に配置するハイブリッドオーケストレーション

開発をキックする
トリガーの拡張

上記開発パイプラインをトリガーする導線の拡充
UI上でアノテーションが可能なChrome拡張や自社プロダクトのAIヒアリングからの自動開発等

self improveへの挑戦

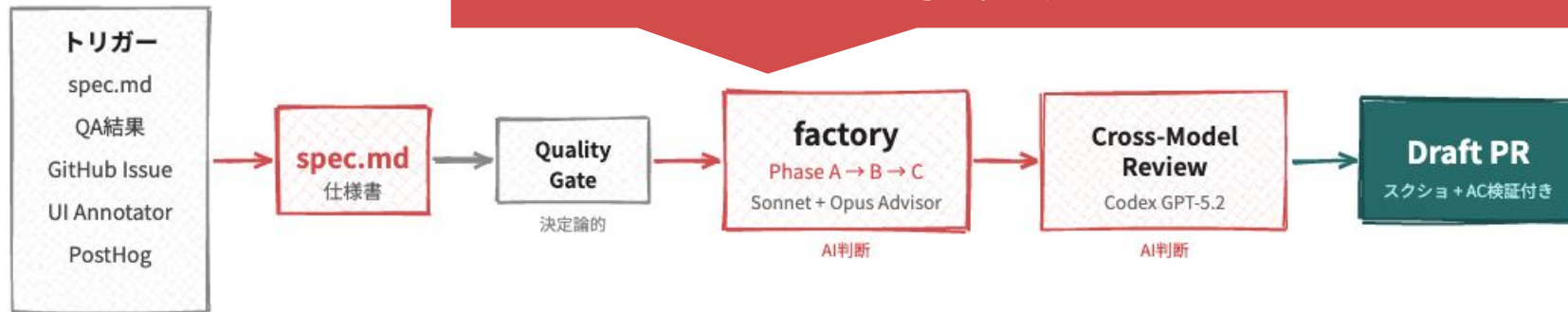
Claude Code/Codexを配り、ルールファイルを整備すればAIネイティブな開発か？
「個人がどれだけうまく使えるか」に閉じていないか？

「行動を制約し、正しいか確かめ、次をもっと良くする」を1つのパイプラインに

Claude Agent SDK + TypeScriptで構築

Claude Code / Codexで人間が行っている開発プロセスをそのまま型化

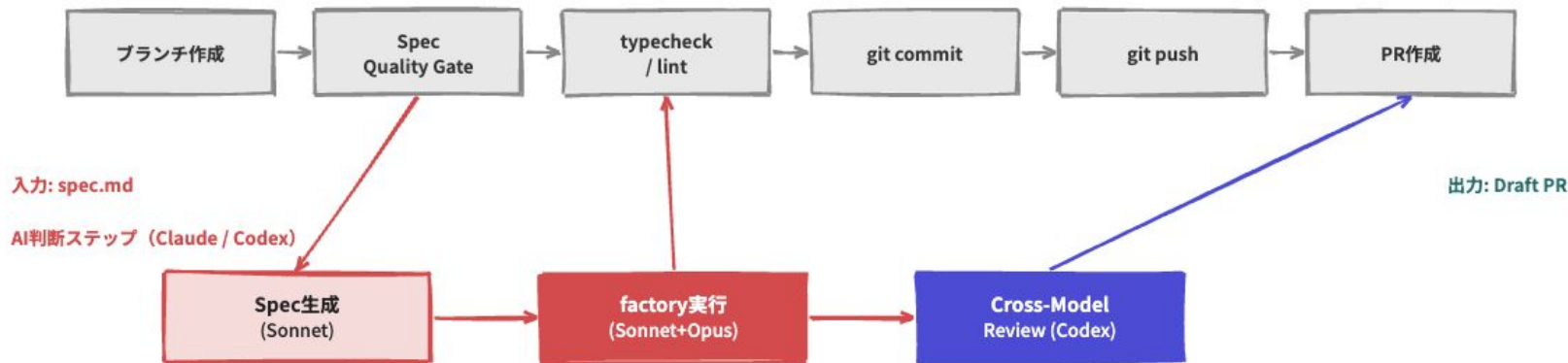
specからブラウザテスト・スクショ付きのPR作成までスクリプト化
CodexによるCross Model ReviewやOpus4.6によるAdvisor
等も組み込み



決定論と確率論のハイブリッドオーケストレーション

スキル・開発用のツール/スクリプトも共通利用してるため、スキルを育てることも、パイプラインの進化にも直結

決定論的ステップ (TypeScript)



図はAI生成によりイメージ

Issueから開発を起動できる

npxコマンドで起動するため、GitHub ActionsでIssueからキックも容易 Issue -> PRが一定のクオリティで担保された上で実現可能

[動画証跡サンプル] アカウント設定で表示切り替えを録画確認する #1683

Open

kazuyaseki opened 6 hours ago

Why

spec-to-pr の動画証跡を CI 環境で確認するためのサンプル issue。
認証後ページで明確な画面変化を起こす最小 UI 変更を題材にして、workflow dispatch 後に生成される sample PR の動画を証跡として確認する。

Requirements

- ☐ <http://localhost:3000/settings/account> のヘッダー付近に 録画サンプルを表示 ボタンを追加する
- ☐ 初期表示では補助パネルは非表示とする
- ☐ 録画サンプルを表示 を押すと補助パネルが表示される
- ☐ 補助パネル内に 録画確認用の補助表示です を表示する
- ☐ 補助パネル内の 閉じる を押すと補助パネルが再び非表示になる
- ☐ 既存のプロフィールカードと二要素認証カードの表示・挙動は変えない

Scenarios

S1: ログイン後に録画サンプルを表示できる

管理画面にログイン済みの利用者がアカウント設定画面を開く。
ヘッダーのボタンを押すと補助パネルが表示され、録画を見た人が「どの操作でどの表示が変わったか」を明確に追える。

S2: 補助パネルを閉じて元の状態に戻せる

Summary

close #1683

- Spec: <docs/dev/2026-04/issue-1683/spec.md>
- Requirements (from spec):
 - R1: <http://localhost:3000/settings/account> のヘッダー付近に 録画サンプルを表示 ボタンを追加する
 - R2: 初期表示では補助パネルは非表示とする
 - R3: 録画サンプルを表示 を押すと補助パネルが表示される
 - R4: 補助パネル内に 録画確認用の補助表示です を表示する
 - R5: 補助パネル内の 閉じる を押すと補助パネルが再び非表示になる

Verify Results — 2026-04-21

AC	Status	Description	Screenshot	Notes
AC-1	OK	アカウント設定画面に「アカウント設定」見出しが表示される		—
AC-2	OK	「録画サンプルを表示」ボタンがヘッダー右端に表示される		—
AC-3	OK	初期状態では「録画確認用の補助表示です」が表示されていない		—

Claude Agent SDK + TypeScriptでAI開発フローをまとめる良さ

開発フロー全体がコードだから、クラウド実行ができ、ログ・トレースも取りやすいソフトウェアと同じように扱える

再現性と属人性の排除

誰が起動しても同じプロセスが走る

個人のAI開発への熟練度に依存しなくなる
今はデザインハーネスの組み込みに着手

品質保証の組み込み

ブラウザテスト・証跡スクショ・動画まで完了した状態でPRが出る

不良specを早期に弾き、Codexクロスモデルレビューで検証

計測と改善の基盤

スクリプトの中で全実行のログが取れる

設計判断・インシデント履歴をDBに永続化させるなど、「効いてるかわからない」を解消する土台も組み込みやすい

ハーネスエンジニアリングにも評価や育てる仕組み

Braintrustにログを送り、分析と改善が回せる仕組みを作っている
定量的な評価やダッシュボードの本格化に着手、どの類のタスクがどの程度の成功率 / 人間から追加介入なしでマージされているか？等を追跡

braintrustのログを元に分析してもらいました

<https://www.braintrust.dev/app/>

Untitled ▼

```
1 # Trace分析レポート:
2
3 **Trace ID**: `99ac059f-ae66-4687-87c7-43337a3425bf`
4 **モデル**: `claude-sonnet-4-6`
5 **実行時間**: 2時間00分 (7,201秒)
```

The screenshot displays the Braintrust interface, which is used for managing and analyzing AI agent interactions. The interface is divided into several sections:

- Logs:** A table showing a list of traces. The columns include 'Created', 'Name', 'Input', 'Output', and 'Expected'. The table lists 12 traces, with the most recent ones from 1h ago and 3h ago.
- Trace tree:** A detailed view of a specific trace, showing the sequence of actions and the output of the AI agent. It includes a 'Trace tree' section with a list of actions and their results.
- Root span:** A section showing the overall performance metrics for the trace, including 'Start', 'Duration', 'Estimated cost', and 'Input'.
- Metrics:** A section showing the performance metrics for the trace, including 'Start', 'Duration', 'Estimated cost', and 'Input'.

The interface is dark-themed and includes various navigation and filtering options.

ハーネスエンジニアリングにも評価や育てる仕組み

Braintrustにログを送り、分析と改善が回せる仕組みを作っている
定量的な評価やダッシュボードの本格化に着手、どの類のタスクがどの程度の成功率 / 人間から追加介入なしでマージされているか？等を追跡

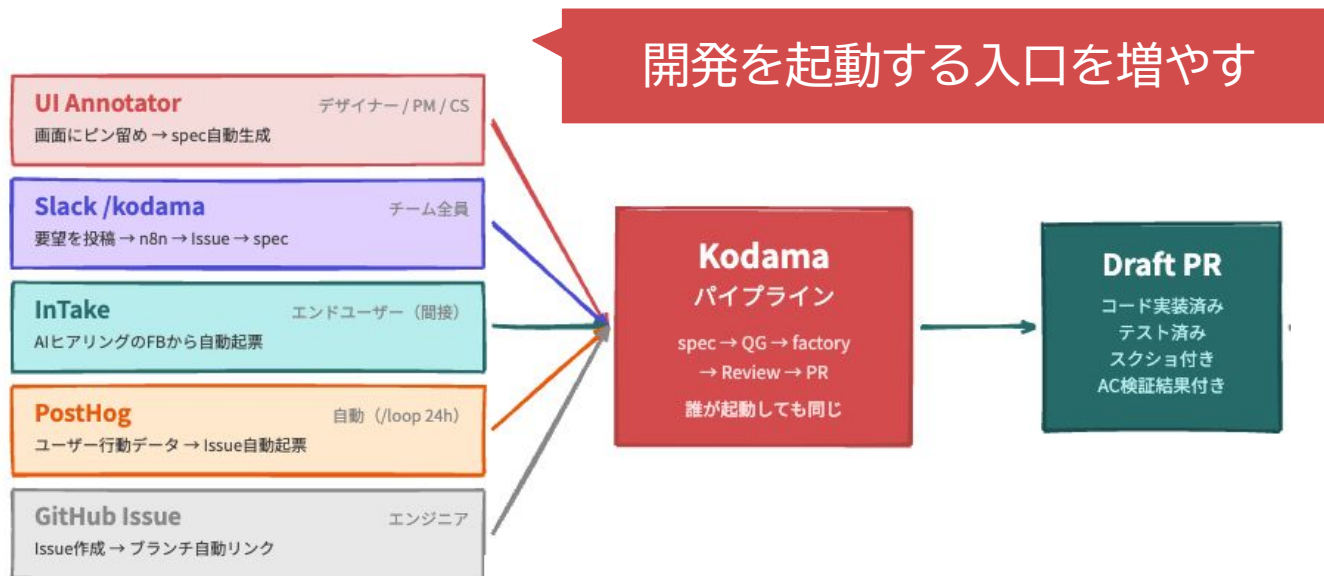
後述のself improveの文脈で、
分析・改善提案から修正まで自動化

```
1 # Trace分析レポート: [REDACTED]
2
3 **Trace ID**: `99ac059f-ae66-4687-87c7-43337a3425bf`
4
5 **モデル**: `claude-sonnet-4-6`
6
7 **実行時間**: 2時間00分 (7,201秒)
```

The screenshot displays the Braintrust interface, which is used for monitoring and improving AI agent performance. It features a 'Logs' section on the left, a 'Trace tree' in the center, and a 'Root span' on the right. The 'Trace tree' shows a sequence of actions performed by the 'Claude Agent', including 'anthropic.messages.create', 'ToolSearch', 'Read', and 'Bash'. The 'Root span' provides a summary of the trace, including the start time, duration, and estimated cost. A 'Metrics' section is also visible, showing various performance indicators. The interface is designed to be user-friendly and informative, allowing users to quickly identify and address issues with their AI agents.

開発をキックするトリガーの拡張

開発パイプラインが作れたら、トリガーを拡張する
エンジニア以外にも、ターミナル以外からも開発をトリガー可能に



画面から開発をトリガーするUI Annotator(Chrome拡張)を作る

画面上でのピン留めやテキスト入力が、前述のパイプラインに流れてPRが作成



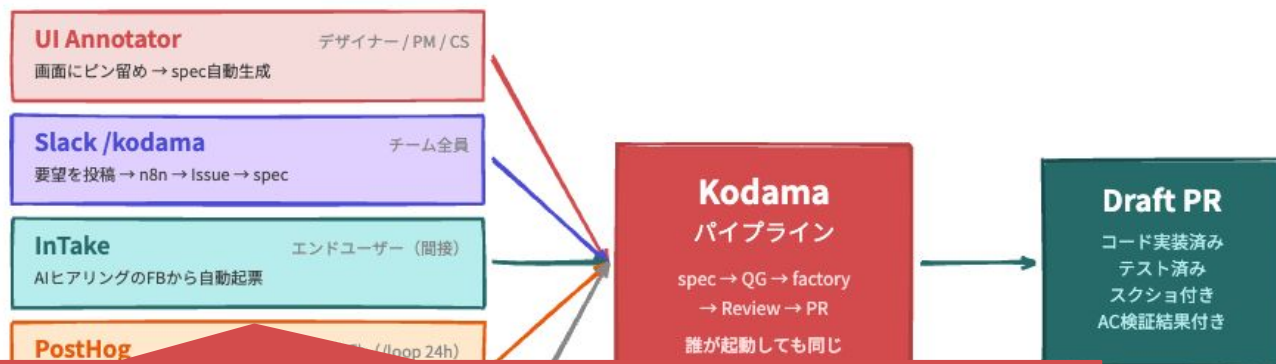
画面から開発をトリガーするUI Annotator(Chrome拡張)を作る

画面上でのピン留めやテキスト入力が、前述のパイプラインに流れてPRが作成



開発をキックするトリガーの拡張

開発パイプラインが作れたら、トリガーを拡張する
エンジニア以外にも、ターミナル以外からも開発をトリガー可能に



変わり種。自社プロダクトでAIがヒアリングした結果をそのまま開発に回す(これは基本ドッグフーディング目的)

Self-improveへの挑戦

ルールファイルの整合性確認・修正等のフィードバックループ自動化
同様にClaude Agent SDKでツールとしてスクリプトを書いて自動実行
直近はエラー履歴のメモリ化に着手

エラー履歴のメモリ化

コード修正で起きたエラーをDBに蓄積し、次の開発サイクルで参照

ルールファイルの自動メンテナンス

定期的にCLAUDE.md/skillsの整合性確認と更新。Claude Agent SDKで自動実行

パイプライン自体の自律的改善

現在は手動（という名のAIエージェント）。パイプライン自体の自律改善に着手

Self-improveへの挑戦

ルールファイルの整合性確認・修正等のフィードバックループ自動化
同様にClaude Agent SDKでツールとしてスクリプトを書いて自動実行
直近はエラー履歴のメモリ化に着手

エラー履歴のメモリ化

コード修正で起きたエラーをDBに蓄積し、次の開発サイクルで参照

ルールファイルの自動メンテナンス

定期的

パイプライン

現在は

せっかく自動ブラウザテストしてるので、「Xを編集したら、Yでエラーが出た」などの履歴を蓄積する
ローカルのSQLiteにエージェントから格納、workするかは定かではない

Self-improveへの挑戦

実際に動かしているClaude Agent SDK製の自動修正系の仕組み 開発パイプライン自体の改善は基本人間 with エージェントで今は頑張っている

check

CLAUDE.md/skillsの整合性を決定論的にチェック。Dead paths・Broken symlinks・Stale refs検出
Rust製、~50ms。hooksで毎回実行。「文書は腐るがlintルールは永続する」

entropy

ハーネスのGC。ドキュメント間の矛盾・陳腐化・コードとの乖離をLLM意味解析で検出
Claude Agent SDK → GitHub Issue自動生成 → Kodamaで修正。蓄積と剪定の自動化

feedback-loop

実行ログ・QAレポート・Entropyレポートを分析し、CLAUDE.md/skillsへの改善提案を自動生成
Cross-session learningのパターンDB。trajectoryから学ぶ自己改善ループ

hisha

知識基盤。設計判断・インシデント履歴を永続化。変更影響範囲分析 (blast-radius)
タスクコンテキスト予測、spec.md自動補強を目指している。最近運用開始

やがてはハーネスの自動生成

Meta-Harness(Stanford)やAutoHarness(Google DeepMind)では、LLM自身が制約を発見し、ハーネスを生成・改善、コード化する



まとめ

今後ハーネスエンジニアリング(的なもの)にどう向き合うか？

モデルやマネージドサービスの進化によって消えづらいレイヤーに投資する
LLMプロバイダーを切り替えても同様に利用可能な仕組みを中心

消えるもの

(モデルの弱点を補う足場)

・計画ステップの強制

(Sonnetで必要→Opusでdead weight)

- ・ Context resets
- ・ Compaction workarounds
- ・ 特定モデル向けのプロンプトハック

保質期: ~1モデル世代 (3-6ヶ月)

残るもの

(モデルが環境と接続するインフラ)

- ・ ワークフロー定義 (factory)
- ・ ドメイン知識 (spec.md、アーキテクチャ)
- ・ 評価の仕組み (テスト、Quality Gate)
- ・ 自己改善ループ (Self-Refinement)

→ ここに投資する

まとめ

- (ここ数年こんな感じだが)定義はさておき、ハーネスエンジニアリングの「モデルの外側の仕組み」を設計する営みは重要
- ルールファイルやスキル整備だけでは足りず、評価・計測・ガードレール・プロセス全体まで射程を広げる必要がある
- AI時代のエンジニアは、コードを書くことに加えて、コードを書く仕組みを設計し育てる役割を担う
- エージェントのログを読み、失敗を分析し、仕組みに還元すること自体が、重要なエンジニアリングスキル

唐突の宣伝で恐縮ですが、6月のFindy主催AI Engineering Summit Tokyo 2026でさらに整理した内容でお届けできる予定です..！



AIE 2026

主催者 ツール 求人情報 スポンサー アクセス X AI

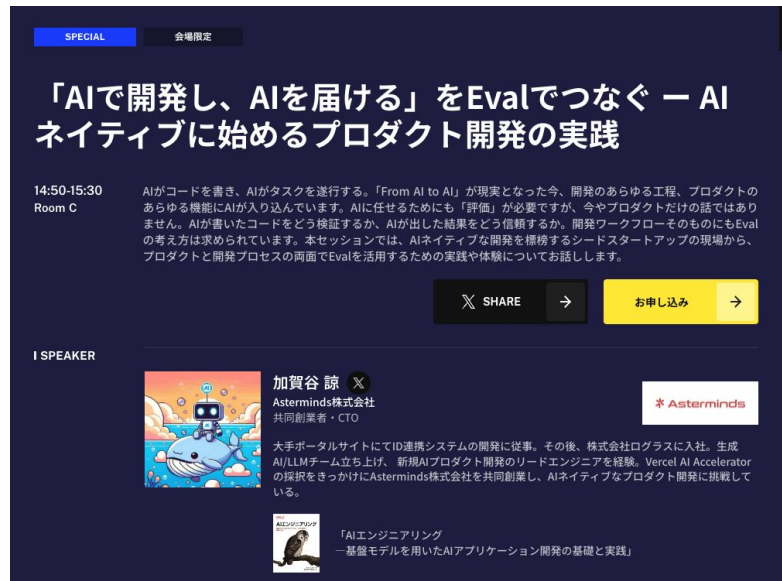
お申し込み タイムテーブル

AI Engineering Summit Tokyo 2026

produced by Findy Tools

2026.6.8(月)9(火) 9:00~21:00
浜松町コンベンションホール & Hybridスタジオ

お申し込み タイムテーブル



SPECIAL 会場限定

「AIで開発し、AIを届ける」をEvalでつなぐー AIネイティブに始めるプロダクト開発の実践

14:50-15:30
Room C

AIがコードを書き、AIがタスクを遂行する。「From AI to AI」が現実となった今、開発のあらゆる工程、プロダクトのあらゆる機能にAIが入り込んでいます。AIに任せるためにも「評価」が必要ですが、今やプロダクトだけの話ではありません。AIが書いたコードをどう検証するか、AIが出した結果をどう信頼するか。開発ワークフローそのものにもEvalの考え方は求められています。本セッションでは、AIネイティブな開発を標榜するシードスタートアップの現場から、プロダクトと開発プロセスの両面でEvalを活用するための実践や体験についてお話しします。

SHARE 申し込み

ISPEAKER

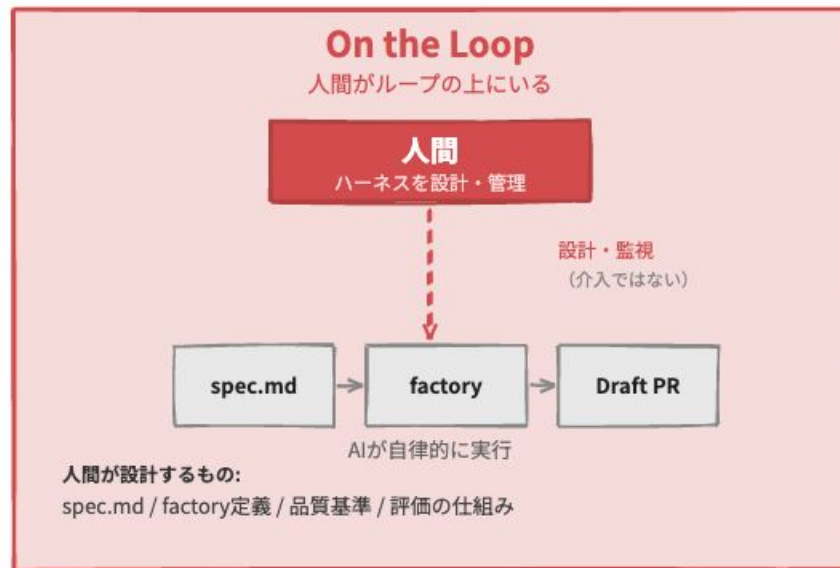
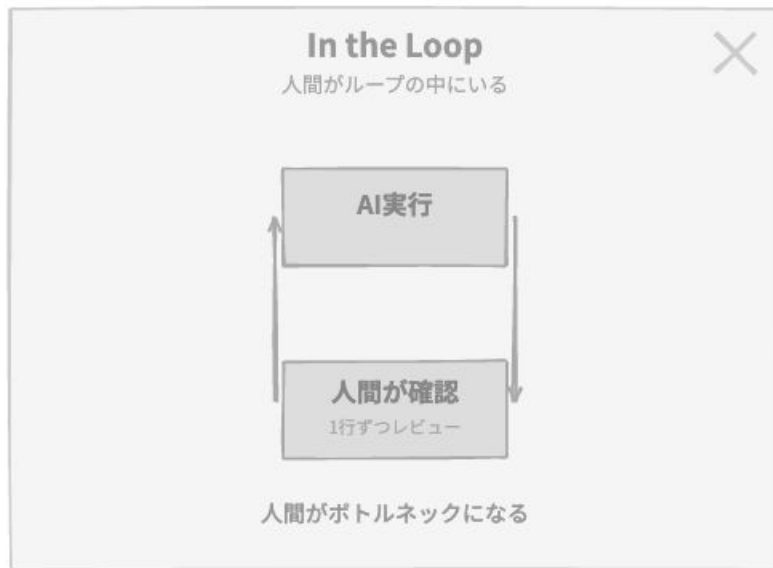
加賀谷 諒
Asterminds株式会社
共同創業者・CTO

大手ポータルサイトにてID連携システムの開発に従事。その後、株式会社ログラスに入社。生成AI/LLMチーム立ち上げ、新規AIプロダクト開発のリードエンジニアを経験。Vercel AI Acceleratorの採択をきっかけにAsterminds株式会社を共同創業し、AIネイティブなプロダクト開発に挑戦している。

「AIエンジニアリング
ー基盤モデルを用いたAIアプリケーション開発の基礎と実践」

今後ハーネスエンジニアリング(的なもの)にどう向き合うか？

人間が介在するタイミングやタッチポイント設計も大事(Human-in-the-loop)
一方、ループの上でハーネス(仕様、品質チェック、ワークフロー)を設計するのが
これからのエンジニアの役割



おわり